Institut Teknologi Bandung

Directorate of Sustainable Professional Education

MODULE

Agentic AI

From the textbook definition of an agent to a system a regulator will accept — what to build, what to buy, and when not to build an agent at all.

Duration 6 hours (4 sessions) **Instructor** Hendri Karisma, M.T.

Source Module material for Designing and Building AI Products and Services

LEARNING OUTCOMES

Session Objectives

By the end of this session, participants can:

1. Trace the word *agent* from **Russell & Norvig** to its current use, and say precisely what has and has not changed.
2. Apply one **operational test** to decide whether a given system is a workflow or an agent — and why most are workflows.
3. Name the **five workflow patterns** and the point at which each stops being enough.
4. Specify the components of an agentic system at three levels: **minimum, best practice, production ready**.
5. Choose a **tech stack** across easy / medium / production tiers, commercial and open source, and justify it on properties you can verify.
6. Judge **no-code platforms** against code on the four things that actually differ.
7. Design both interfaces: **machine-to-machine** (MCP, A2A, REST, events) and **human-to-machine** (approval, escalation, audit).
8. Map a deployment against **UU PDP, OJK, ISO/IEC 27001:2022 and 27701:2025**, and produce the evidence an auditor asks for.
9. Produce a **system design and a deployment design**, including where the data is allowed to be.
10. Walk one end-to-end case: a Flutter app on Android and iOS, a REST service, a classical ML model behind MCP, and a human who makes the decision.

SITE
Course home

REPO
ai-agentic-demo — single-agent and multi-agent cases

BOOK
Verified reference list (books, papers, standards, regulation)

PAPER
OJK — Tata Kelola Kecerdasan Artifisial Perbankan Indonesia (2025)

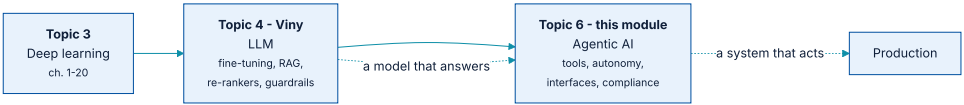
01

Where this module sits

The LLM module ended with a model that answers. This one is about a system that acts.

POSITIONING

This continues directly from the LLM module



Answering is not acting. Everything in this module is about the gap between the two.

Nothing here makes the model more capable. Everything here is about **what the model is connected to**, and about **what happens when it is wrong**.

POSITIONING

What you already have, and what is still missing

FROM THE LLM MODULE	STILL MISSING
A model tuned to your domain	A way for it to **do** anything
Retrieval over your documents	Retrieval as a **choice the model makes**, not a fixed step
Guardrails on input and output	A guardrail on the **action** — the one that moves money
A good answer	A record of **how** it got there, that an auditor accepts

> The last row is the one that decides whether any of this reaches production in a regulated institution.

02

What an agent is

The word is older than the hype, and the old definition is the useful one.

The definition that predates all of this

> *An **agent** is anything that can be viewed as perceiving its environment through **sensors** and acting upon that environment through **actuators**.*
>
> Russell & Norvig, Artificial Intelligence: A Modern Approach, 4th ed., ch. 2

The agent function maps a percept history to an action. Nothing in it requires a language model.

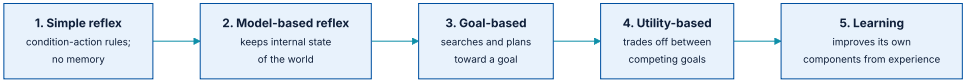
A thermostat satisfies this. So does a list-sorting program. **The definition is broad on purpose** — which is why it needs the next slide.

PEAS: how to specify one before building it

	MEANING	OUR CREDIT CASE
Performance	How is success measured?	Correct recommendation rate; time to decision; zero unapproved writes
Environment	What does it operate in?	Core banking APIs, transaction history, policy corpus, a field officer
Actuators	What can it do?	Read data, run a model, retrieve policy, **draft** — never approve
Sensors	What can it perceive?	Application form, customer record, 12 months of transactions

If you cannot fill the P row with a number, you do not yet have a project — you have an intention.

Five agent types, and where LLM agents actually sit



The classical taxonomy, in increasing order of sophistication.

A tool-calling LLM agent is **goal-based**: it searches over actions toward a stated goal. It is not **utility-based** — it has no explicit utility function to trade off competing goals — and it is not a **learning agent**: it does not improve its own components from experience within a run.

DEFINITION

The modern definition, stated carefully

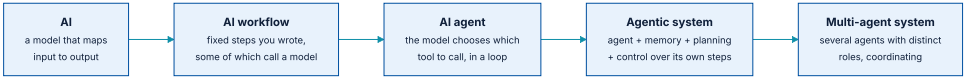
An **agentic system** is one in which a model decides, at run time, which actions to take and in what order, against tools it did not choose and within limits it cannot change.

Three clauses, each doing work:
decides at run time — separates it from a workflow.
tools it did not choose — you supply the action space; that is your control surface.
limits it cannot change — budgets, permissions and approval gates sit outside the model.

> Notice what is *not* in the definition: intelligence, understanding, reasoning, or autonomy as a virtue. **Autonomy is a cost, purchased when flexibility is worth more than predictability.**

VOCABULARY

Five terms, in increasing order of autonomy



Each step gives the model more say over what happens next — and costs more to test.

Vendor material uses these interchangeably. They are not interchangeable: the difference between them is a **cost** difference, a **testability** difference, and a **risk** difference.

WHICH IS WHICH

Agentic, or not? Six systems

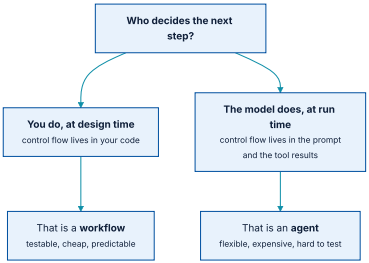
SYSTEM	AGENTIC?	WHY
A chatbot answering from a RAG index	**No**	One fixed retrieve-then-answer path. No decision about what to do.
A thermostat	**No**	An agent by the textbook definition, but a simple-reflex one; no goal, no choice.
Document classifier routing to one of five queues	**No**	That is routing — a workflow pattern. The path set is fixed.
Model that decides whether to search, then which of nine tools	**Yes**	The control flow is chosen at run time from a supplied action space.
Coding assistant that runs tests, reads failures, retries	**Yes**	Observes its own results and revises. This is the loop.
n8n flow calling an LLM at step 3 of 7	**No**	A workflow with a model in it. Nothing wrong with that — it is usually the right answer.

03

Workflow versus agent

One question separates them, and it decides your cost, your tests, and your risk.

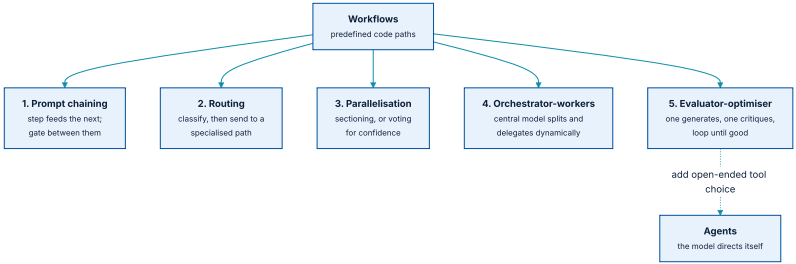
Who decides the next step?



Everything else — the model, the tools, the prompt — can be identical.

If you can draw the flowchart in advance and it does not change per request, you have a workflow. Build it as one. It will be cheaper, faster, and you will be able to test it.

Five workflow patterns before you reach for an agent



Workflows orchestrate models through predefined code paths; agents let the model direct itself.

The published guidance is explicit: **start simple, and add agency only when flexibility outweighs latency, cost, and compounding error.** It also warns against reaching for a framework first, because the abstraction hides the prompts and responses you need to debug.

When each pattern is the right answer

PATTERN	USE IT WHEN	STOPS WORKING WHEN
Prompt chaining	The task decomposes into fixed sequential steps	The steps depend on what earlier steps found
Routing	Inputs fall into known categories needing different handling	A new category appears, or one input needs two paths
Parallelisation	Subtasks are independent, or you want several votes	The subtasks need each other's results
Orchestrator-workers	Subtasks cannot be known in advance	The orchestration itself becomes the hard part
Evaluator-optimiser	You have a clear criterion and iteration helps	The evaluator is as unreliable as the generator

The last row is the one to be careful with: an LLM critiquing an LLM inherits the same blind spots. **Prefer a deterministic evaluator whenever the criterion can be expressed as code.**

THE ECONOMICS

What autonomy actually costs

	WORKFLOW	AGENT
Model calls per request	Known — 1 to 7	**Unbounded until you bound it**
Latency	Predictable	Varies by an order of magnitude
Cost per request	Computable in advance	A distribution, with a long tail
Testing	Ordinary integration tests	Evaluation sets, and they are never complete
Failure mode	An exception you can catch	A plausible wrong answer
Audit story	The code *is* the flowchart	The trace is the only record

> Every row favours the workflow. That is not an argument against agents — it is the **price list**, and you should know it before you pay.

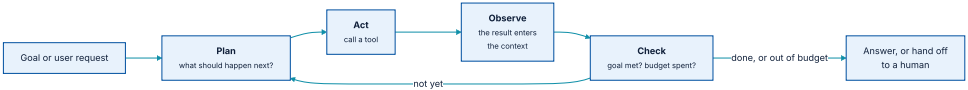
04

How an agent runs

A loop, a tool schema, four kinds of memory, and something that makes it stop.

THE LOOP

Plan, act, observe, check



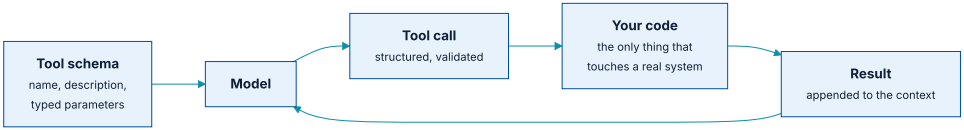
The whole of agent engineering is in what fills each box.

This is the **ReAct** pattern — reason, then act, then read the result and reason again. Published as an ICLR 2023 paper and now assumed by every framework in the field.

Six ways it must be able to stop

- **Goal satisfied** — the model says it is done, and something else checks that claim.
- **Step budget** — a hard maximum number of iterations.
- **Token or currency budget** — a hard maximum spend per request.
- **Wall-clock timeout** — a hung tool is not the model's problem to solve.
- **Repetition detector** — the same tool with the same arguments twice running is a loop, not progress.
- **Escalation path** — what happens when the budget is spent and the goal is not met. *Silently returning a partial answer is the worst option*, because it is indistinguishable from a complete one.

The model never touches anything



The model emits a structured request. Your code decides whether to honour it.

This is the most important architectural fact in the module. **The model produces text describing an intention. Your code is the only thing that executes.** Every permission boundary you have lives in that gap.

A tool is an API for a reader who forgets everything

1. **Name it for the task, not the system.** `find_customer_by_nik` beats `crm_query_v2`.
2. **Write the description for the model.** It is the only documentation the model gets, and it is re-read on every call.
3. **Type every parameter and validate on arrival.** Assume the arguments are adversarial, because sometimes they are.
4. **Return errors as data, not exceptions.** `{"error": "no customer with that NIK"}` lets the agent recover; a thrown exception ends the run.
5. **Keep results small.** Every byte a tool returns occupies context the model pays to re-read on every later step.

Most *prompt engineering* problems in agent systems are **tool description problems** wearing a disguise.

Read tools and write tools are not the same risk

Read tools

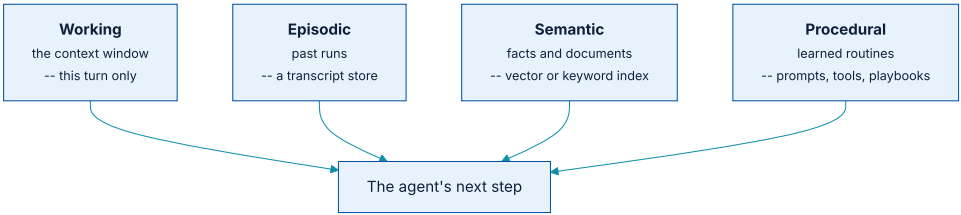
Search, retrieve, look up, score. A wrong call wastes tokens and returns something unhelpful. **Recoverable inside the loop.**

Write tools

Send, delete, transfer, approve, disburse. A wrong call **changes the world**, and no amount of later reasoning undoes it.

Treat them differently from day one. **Read tools can be autonomous; write tools start behind an approval gate** and only leave it with evidence from your evaluation harness. OWASP names the failure: **excessive agency**.

Four kinds, solving four different problems



Four categories, four pieces of infrastructure.

The mistake to avoid is treating *memory* as one thing and reaching for a vector database by reflex. **Most agents need working memory managed well and nothing else.**

What each one costs you to run

KIND	IMPLEMENTATION	THE FAILURE IT INTRODUCES
Working	The context window itself	Overflow — old steps fall out and the agent forgets its goal
Episodic	Transcript store, keyed by session	Replaying stale state as if it were current
Semantic	Vector or keyword index — the LLM module's RAG	Retrieving confidently irrelevant context, which is worse than none
Procedural	Versioned prompts, tool sets, playbooks	Drift between what is deployed and what was evaluated

Each row is a system to operate, back up, and retain lawfully. **Add them one at a time, and only when a specific observed failure demands it.**

05

Single agent and multi-agent

Three questions before you split, and an honest cost table.

The supervisor pattern

This is the pattern in almost every framework, under various names. The variations — peer-to-peer messaging, hierarchies, debate — are **elaborations of it, and each elaboration multiplies the cost**.

> Splitting for conceptual tidiness is the common mistake. Roles named *Researcher*, *Writer* and *Critic* feel organised and typically cost three times one agent doing the same work with the same tools.

Three questions before you split

1. Do the subtasks need **genuinely different tools or permissions**? Then splitting is a *security boundary* — the strongest reason there is.
2. Can they run **in parallel**? Then the latency win is real, provided each subtask is slow enough to hide the coordination overhead.
3. Would one context window **overflow** otherwise? Then splitting is a *memory strategy* and the summaries are the compression.

> **If none of the three is a yes, you want one agent with more tools.** Cheaper, faster, and testable.

What each additional agent costs

COST	WHY IT GROWS
Tokens	Every hand-off restates context. A supervisor summarising for three specialists pays for that summary three times, plus its own reasoning.
Latency	Sequential hand-offs add up. Parallel ones help only if the subtasks are genuinely independent.
Failure surface	Each agent can loop, drift or hallucinate independently — and **one specialist's wrong answer becomes another's trusted input**.
Evaluation	Per-agent evals *and* end-to-end evals, because a system can be right overall for the wrong reasons.
Audit	Every hand-off is a data flow. In a regulated setting each one must be justified and logged.

Split on evidence, not on tidiness.

Three shapes where splitting is right

Different permissions

One agent may read customer records; another may write to the ledger. Splitting is how those capabilities stay apart — a **security boundary**, not a style choice.

Genuine parallelism

Twenty documents to analyse independently. Fan out, collect, assemble. The contexts never interact.

Context that will not fit

Inputs that genuinely exceed the window. Splitting is a **memory strategy**, and the supervisor's summaries are the compression.

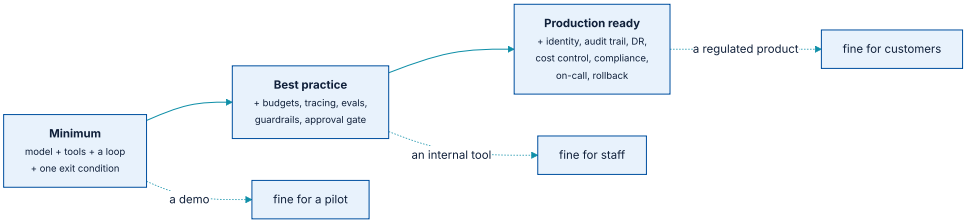
In the credit case later in this module, exactly one of these applies — and it is the first.

06

Components, at three levels

Minimum, best practice, production ready — and the honest label for each.

Three levels, and what each is actually fit for



The label on the right is the honest one. A demo is not an internal tool, and an internal tool is not a product.

The common failure is shipping a **minimum** system with **production** language attached to it. The components are the difference, and they are countable.

LEVEL 1

Minimum — what makes it an agent at all

- **A model** with tool-calling.
- **A tool set** with typed, validated parameters.
- **A loop** that feeds results back in.
- **One exit condition** — usually a step cap.

That is roughly two hundred lines. It will demo well, and it will surprise you the first time a tool returns something unexpected.

> Fit for: **a pilot you watch while it runs.** Not fit for anything unattended.

LEVEL 2

Best practice — what makes it survivable

COMPONENT	WHAT IT PREVENTS
Budgets (all six)	The loop that bills by the token
Tracing	Being unable to answer *why did it do that*
Evaluation sets	Shipping a prompt change that quietly regresses
Guardrails	Injection from retrieved content; secrets in the output
Approval gate on writes	The irreversible action nobody authorised
Structured errors	A run that ends instead of recovering

> Fit for: **an internal tool used by trained staff** who can tell when it is wrong.

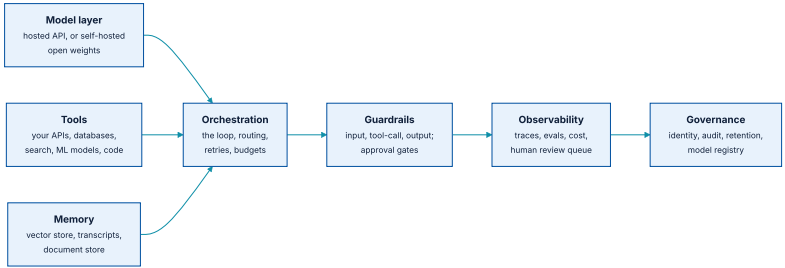
LEVEL 3

Production ready — what a regulator and an on-call rota require

COMPONENT	WHY IT BECOMES MANDATORY
End-user identity propagation	Permissions checked against the person, not the service account
Immutable audit trail	The trace becomes evidence, so it must be tamper-evident and retained
Data classification and retention	UU PDP: you must know what personal data went where, and delete it on schedule
Model registry and versioning	Which model, which prompt, which tool set produced this decision
Cost control and quotas	Per-tenant, not just global
Rollback	Prompts and tool sets are releases; they need to go backwards
Incident runbook and on-call	Someone has to own it at 2 a.m.
DR and residency	Where it runs, and where it fails over to

ARCHITECTURE

Seven layers, and the two that get skipped



Guardrails and observability are the two most first attempts leave out entirely; governance is the one regulated work adds.

Read the middle of that diagram carefully. **The orchestration layer is ordinary distributed-systems work** — queueing, retries, budgets, telemetry — and your existing platform team already knows how to do it.

07

Tech stack

Easy, medium, production — commercial and open source, and what each tool is actually for.

CHOOSING

Three tiers, and the question each answers

Easy

Can this idea work at all? Days to a demo. Hosted everything. Accept the lock-in; you are buying information, not building a system.

Medium

Will this survive real users? Weeks. Your own repo, an SDK, your own tools. Testable and diffable.

Production

Will this survive an auditor? Months. Identity, residency, audit trail, DR, cost control. The model is the small part.

Do not start at the third tier. Most ideas die at the first, and the cheapest place to learn that an idea is wrong is where it costs days.

MODEL LAYER

Hosted or self-hosted, and what actually decides it

	HOSTED API	SELF-HOSTED OPEN WEIGHTS
Examples	Anthropic Claude, OpenAI, Google Gemini, AWS Bedrock	Llama, Qwen, Mistral, Gemma — on vLLM, Ollama, TGI
Capability	Highest available, immediately	Lower at the same price point, closing
Cost shape	Per token — scales with usage	Fixed infrastructure — scales with capacity
Data	**Leaves your perimeter**	**Stays inside**
Effort	An API key	You now operate a GPU fleet

> In practice the decision is rarely capability or cost. It is **where the data is allowed to be** — a regulatory question, answered before any code is written.

What each framework is actually for

TOOL	WHAT IT DOES	TRADE-OFF
No framework	You write the loop — about 200 lines	Most control, most legible. Anthropic's own guidance recommends starting here.
LangGraph	Graph-structured state machine over agent steps	Explicit control flow, persistence, human-in-the-loop. Concepts to learn.
OpenAI Agents SDK	Lightweight loop, handoffs, guardrails	Simple; closest to the provider's own model behaviour.
CrewAI	Role-based multi-agent teams	Fast to a multi-agent demo; encourages splitting before you have a reason.
AutoGen	Conversational multi-agent, research lineage	Strong for experiments; a published architecture to argue with.
LlamaIndex	Retrieval-first, then agents over it	Best when the problem is mostly documents.

One row per layer, with the open-source option named

LAYER	COMMERCIAL	OPEN SOURCE	WHAT IT IS FOR
Vector store	Pinecone, Weaviate Cloud	pgvector, Qdrant, Milvus	Semantic memory and RAG
Tracing / evals	LangSmith, Braintrust, Langfuse Cloud	**Langfuse**, Phoenix, OpenLLMetry	The trace, and regression gates
Gateway	Portkey, Kong AI	LiteLLM, Envoy AI Gateway	One endpoint, key rotation, quotas, failover
Serving	Bedrock, Vertex, Azure AI	vLLM, TGI, Ollama	Running weights you host
Guardrails	Azure AI Content Safety, Bedrock Guardrails	NeMo Guardrails, Guardrails AI	Input, output, topic control
Workflow	Temporal Cloud, Step Functions	**Temporal**, Airflow, Prefect	Durable execution, retries, long-running state

The bolded ones are the two worth knowing regardless of budget: **Langfuse** because traces are non-negotiable, and **Temporal** because a long-running agent is a durable-execution problem before it is an AI problem.

TRADE-OFFS, STATED PLAINLY

Commercial against open source

Commercial / managed

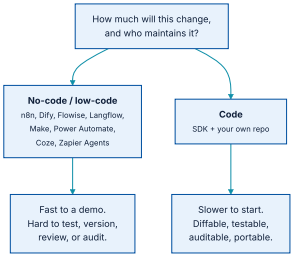
For: fastest to value, support contract, someone else is on call, compliance certifications you can point at.
Against: per-seat or per-token cost that grows with success, data leaves your perimeter, migration cost rises with adoption.

Open source / self-hosted

For: data stays inside, no per-token cost, auditable end to end, no vendor can deprecate you.
Against: you are the support contract, you are on call, and the certifications are now your evidence to produce.

For a regulated Indonesian institution the honest default is **hybrid**: managed model access through a gateway you control, everything stateful — traces, vectors, documents, the ML models — self-hosted inside the perimeter.

No-code platforms, and the four things that actually differ



The question is not which is better. It is how much this will change, and who maintains it.

Worth naming so the room can evaluate them: **n8n, Dify, Flowise, Langflow, Coze, Make, Zapier Agents, Microsoft Power Automate / Copilot Studio, Google Vertex AI Agent Builder**, and the agent-builder features now inside most CRM suites.

Where each one wins

	NO-CODE PLATFORM	CODE
Time to demo	**Hours.** Genuinely.	Days
Who can build it	**A business analyst**	An engineer
Version control	A JSON export, if you are lucky	**Git, with reviewable diffs**
Testing	Manual, mostly	**Automated, in CI**
Audit evidence	Screenshots of a canvas	**Traces, commits, eval runs**
Cost at scale	Per-run pricing that surprises you	Your own infrastructure

For a **regulated** deployment the audit row usually settles it. You cannot show an examiner a canvas and call it a change-control record.

Use both, for different phases

1. **Prototype in no-code.** Find out whether the workflow is even right, with the business owner in the room changing it live.
2. **Write the evaluation set from the prototype.** The twenty real cases it got wrong are worth more than the prototype itself.
3. **Rebuild the survivor in code**, against those evaluations, once you know the shape is stable.
4. **Keep the no-code tool** for the genuinely low-risk automations that will never face an auditor.

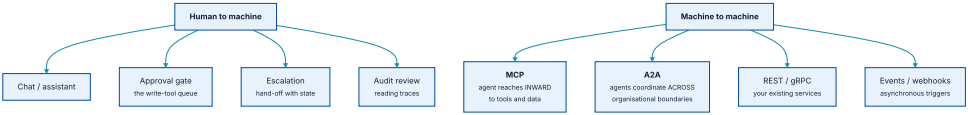
This is not a compromise. Prototyping is a **different activity** from building, and using one tool for both is what makes prototypes accidentally become production.

08

Interfaces

Machine to machine, and human to machine. Both need designing.

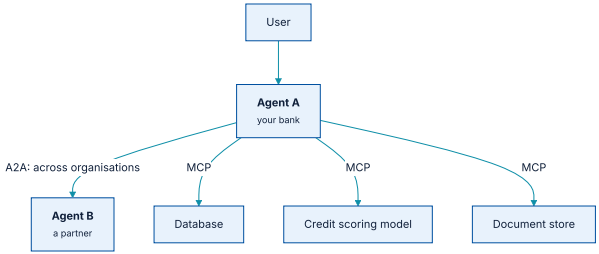
Two directions, four kinds each



The human column is the one that gets designed last and matters most in a regulated setting.

Machine-to-machine is the half everyone builds: MCP inward to tools, A2A across organisations, REST and events to everything already running. **Human-to-machine** is the half that gets skipped — approval, escalation, audit — and it is the half a supervisor reads.

MCP and A2A do different jobs



MCP reaches inward to tools and data; A2A reaches across to other organisations' agents.

MCP — introduced by Anthropic in November 2024, now under the Linux Foundation. JSON-RPC, modelled on the Language Server Protocol. The 2025-06-18 revision added structured tool output, elicitation, and classified MCP servers as OAuth **Resource Servers** with RFC 8707 resource indicators — which is what makes it usable under a real identity model.

A2A, and why a standard mattered here

	MCP	A2A
Direction	Agent → tools and data	Agent ↔ agent
Boundary	**Inside** your system	**Across** organisations
Analogy	A driver interface	A negotiation protocol
In our case	Customer API, credit model, analytics, policy	Not used — one institution, one agent

> Do not adopt A2A because it exists. **It solves cross-organisational coordination**, and if you have one organisation you have added a protocol and bought nothing.

Four interfaces, and the one nobody designs

Chat / assistant

The one everybody builds. Fine for exploration; a poor fit wherever the task has a fixed shape and a form would be faster.

Approval gate

Where a write tool waits. Must show **what** will happen, **on what evidence**, and offer a real decline. This is a product surface, not a dialog box.

Escalation

The hand-off nobody designs. Must carry what was already established so the human does not repeat the work — and must come back.

Audit review

Someone reads traces: sampled, plus every escalation. If the trace is unreadable by a human, it is not an audit trail.

Designing the approval gate properly

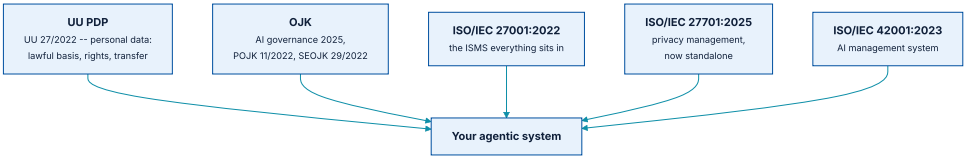
- **State the action in the approver's language**, not the tool's. *Credit IDR 250,000 to account acc-771*, not `post_credit(...)`.
- **Show the evidence** the agent used, with links to the source records.
- **Show what it did not check.** An approver needs to know the boundary of the analysis.
- **Make decline as easy as approve**, and record the reason — that is your best training data.
- **Never batch approvals by default.** Batching is how a gate becomes a rubber stamp.

09

Regulation and compliance

UU PDP, OJK, ISO/IEC 27001 and 27701 — and a strategy for satisfying all of them at once.

Five instruments your system sits inside



They overlap heavily. That is good news: one control usually satisfies several.

Everything on this and the following slides was read from the primary documents. The full citation list, with the PDFs where they are downloadable, is in `references/REFERENCES.md` in the slides repository.

OJK · APRIL 2025

Tata Kelola Kecerdasan Artifisial Perbankan Indonesia

CHAPTER	WHAT IT GIVES YOU
1	Background — where AI already sits in Indonesian banking
2	**Risks and challenges** — the risk register to start from
3	Regulation benchmark across countries — EU, Singapore, US
4	**Guiding principles** — the values every control maps back to
5	**Risk management and governance** — roles and three lines
6	**Implementation guidance** — the lifecycle, stage by stage
7	**Supervision and audit** — what an examiner will ask for

> OJK positions it as a **minimum reference**, explicitly flexible as standards evolve. Read: it is a floor, not a ceiling.

The principles, read out of the document

Four ethics principles

Respect for human autonomy · Prevention of harm · Fairness · Explicability.
(EU High-Level Expert Group lineage.)

Nine governance principles

Proportionality and Do No Harm · Safety and Security · Right to Privacy and Data Protection · Adaptive and Collaborative Multi-Stakeholder Governance · Responsibility and Accountability · Transparency and Clarity · Human Supervision and Control · Sustainability · Awareness and Literacy.
(UNESCO Recommendation lineage.)

The document also benchmarks **MAS FEAT** (Singapore) and uses the **ISACA AI Audit Toolkit** and the **VCIO model** for its audit chapter.

The AI guidance sits on top of existing obligations

INSTRUMENT	WHAT IT ALREADY REQUIRES OF YOU
POJK 11/POJK.03/2022	Penyelenggaraan Teknologi Informasi oleh Bank Umum — IT governance, risk management, third-party and cloud arrangements
SEOJK 29/SEOJK.03/2022	Ketahanan dan Keamanan Siber — cyber resilience and security
SEOJK 24/SEOJK.03/2023	Digital maturity assessment for commercial banks
POJK 1/POJK.03/2019	Internal audit function — who has to be able to review this
POJK 3/2024	Financial-sector technology innovation, including the sandbox

Nothing in the AI guidance replaces these. An agentic system is an IT system first: it inherits every obligation your core banking platform already carries.

What personal data protection means for an agent

THE QUESTION	WHERE IT LANDS IN YOUR DESIGN
What is your **lawful basis**?	Decided per data class, before the tool is written
Does personal data reach the **model provider**?	Redact at the tool boundary, or self-host. This is the decision that shapes the whole stack.
Can you honour **erasure**?	Traces contain personal data. They need retention rules and a deletion path, like any other store.
Can you explain an **automated decision**?	Reason codes from the model, plus the trace. This is why the credit model is classical ML, not an LLM.
Where does it **cross a border**?	Every egress, including the model API call, is a transfer

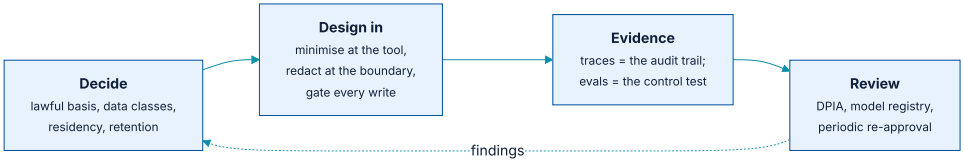
One correction worth making now

> **ISO/IEC 27701 changed.** The second edition was published on **14 October 2025** and is now a **standalone management system standard**. The 2019 first edition was an *extension* to 27001 and 27002 — and a great deal of material still in circulation describes it that way.

Also worth knowing: **ISO/IEC 42001:2023** (AI management systems) and **ISO/IEC 23894:2023** (AI risk management guidance) are the AI-specific companions, and their vocabulary maps closely onto the **NIST AI RMF** Govern / Map / Measure / Manage structure.

STRATEGY

Satisfying all of them without doing the work five times



One loop, four stages. The instruments differ; the evidence they want is largely the same.

The insight that saves the most effort: **your traces are already the audit trail, and your evaluation sets are already the control tests.** Build them for engineering reasons, and the compliance artefacts are a by-product rather than a second project.

One control, several obligations

CONTROL YOU BUILD	SATISFIES
Immutable trace of every run	OJK ch. 7 (audit) · ISO 27001 A.8.15 logging · UU PDP accountability · NIST *Measure*
Approval gate on write tools	OJK human supervision and control · ISO 27001 A.5.15 access control · UU PDP automated-decision safeguards
End-user identity propagation	ISO 27001 A.5.15 / A.8.2 · POJK 11/2022 IT governance
Redaction at the tool boundary	UU PDP minimisation and transfer · ISO 27701 PII controls
Evaluation set as a release gate	OJK ch. 6 lifecycle · ISO 42001 · NIST *Manage*
Model and prompt registry	OJK explainability and accountability · ISO 42001 · change control

Six controls, roughly twenty obligations. **Design the control once, map it many times** — and keep the map, because that table is what you hand an examiner.

The question to settle before writing any code

May customer personal data leave the perimeter to reach a model provider?

Every architectural decision in this module follows from that answer, and it is not an engineering decision.

IF THE ANSWER IS…	THEN YOUR ARCHITECTURE IS…
No, never	Self-hosted open weights inside the perimeter. Lower capability, fixed cost, full control. Plan the GPU capacity.
Only if de-identified	Hosted model, with redaction and tokenisation **at the tool boundary** — and a test proving nothing personal crosses it.
Yes, under contract	Hosted model with a data processing agreement, in-region processing, zero-retention terms, and the transfer documented under UU PDP.

> Ask it in the first meeting. Teams that defer it rebuild.

10

System design

One case, designed properly: where each component sits and why.

THE CASE

SME credit assessment, in the field

Real data analysis
Twelve months of transaction history, aggregated and compared against the applicant's stated turnover.

A classical ML model
Credit scoring — gradient-boosted trees, **not an LLM** — exposed as a tool. It returns a score *and reason codes*.

Regulation in the loop
UU PDP on the customer data, OJK on the decision, and a policy corpus the agent must cite from.

A mobile front end
A Flutter app — Android and iOS — the field officer uses at the applicant's premises, often on a poor connection.

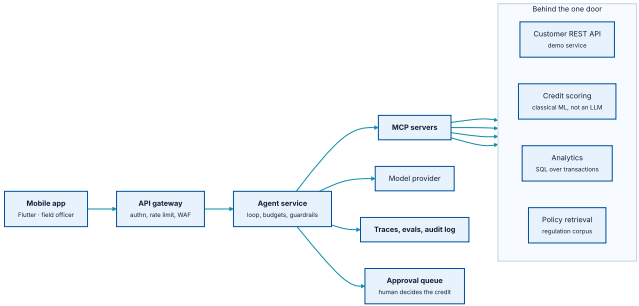
The agent recommends. A human decides.

The agent has no tool that approves credit. Not a disabled one, not a gated one — the capability is absent from its registry.

That is a deliberate architectural choice with three justifications, and it is worth being able to state all three:

1. **Regulatory** — OJK's human supervision and control principle, and UU PDP's safeguards around automated decisions with legal effect.
2. **Technical** — an LLM cannot compose functions reliably, and a credit decision is a composition. *Faith and Fate* (NeurIPS 2023) is the published evidence.
3. **Practical** — an instruction can be argued with; an absent tool cannot. Removing the capability is stronger than forbidding its use.

The components, and what talks to what



Four MCP servers, one of which is a classical ML model. The approval queue is a first-class component, not an afterthought.

Note what the agent service does **not** hold: no customer data at rest, no credentials for the core systems. It holds a loop, a budget, and a set of capabilities it was granted.

Five read tools, one write, and why they are separate

TOOL	KIND	WHY SEPARATE
`get_customer`	read	Wraps the existing REST service. Different owner, different release cycle, different rate limits.
`analyse_transactions`	read	SQL over the warehouse. Heavy, cacheable, and needs its own timeout.
`score_credit`	read	**Classical ML behind an API.** Versioned separately, monitored for drift, and auditable on its own terms.
`retrieve_policy`	read	The regulation and internal-policy corpus. Its own index, its own update cadence.
`check_policy`	read	Runs every machine-checkable clause, returns pass/fail. **Policy is deterministic and belongs in code.**

All five are read tools. There is exactly one write in the whole system, and it is on the next slide.

One write tool, and it writes to a queue

`submit_recommendation` — queues a recommendation for a named officer to decide. **Nothing here approves credit.**

1. **The capability does not exist.** There is no `approve_credit` anywhere in the registry, so no prompt can reach one and no injected instruction can talk the model into one.
2. **A test asserts it.** `test_the_agent_has_no_tool_that_approves_credit` fails if a write tool other than this one ever appears. The claim on this slide is checked on every commit.
3. **The server fills in the provenance itself.** The model supplies a recommendation and a rationale; the *score*, the *model version* and the *policy result* are re-derived server-side. A field the model could have misremembered is a field an auditor cannot rely on.
4. **A decision needs a person and a reason.** The queue refuses a decision with no officer id, refuses an empty reason, and refuses to let the same item be decided twice.

Why the score comes from a classical model, not the LLM

	CLASSICAL ML	LLM
Determinism	**Same input, same score**	Varies by sampling
Explainability	**Reason codes, SHAP values**	A plausible narrative, unverifiable
Validation	Standard model risk practice, backtesting	No accepted method for a credit decision
Drift monitoring	**Well understood**	An open problem
Regulatory acceptance	**Established**	Not established
Cost per call	Fractions of a cent	Orders of magnitude more

The LLM's job here is **orchestration and explanation** — deciding what to look up, and writing the recommendation in the officer's language. The numbers come from something that can be validated. **Use each for what it is good at.**

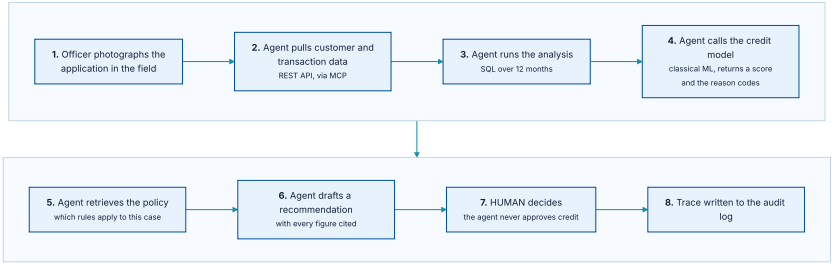
Which classical model? We measured, and it was not the fancy one

CANDIDATE	HELD-OUT AUC	VERDICT
Gradient-boosted trees	0.733	Rejected. Spent its capacity on noise.
Logistic regression	**0.762**	**Shipped.** Also the one you can put in front of an examiner.

The relationship in this data is close to linear and the population is small, so the ensemble had nothing to find and found noise instead. **Reaching for the most powerful model is a habit, not a method.** Here the simpler model wins twice: on the metric, and on **being explainable to somebody who can stop you from deploying it.**

> Run `python3 integrated/run_demo.py --check` and the comparison is printed. If your data is different, you will get a different answer — which is the point of running it rather than assuming it.

Eight steps, from photograph to audit log

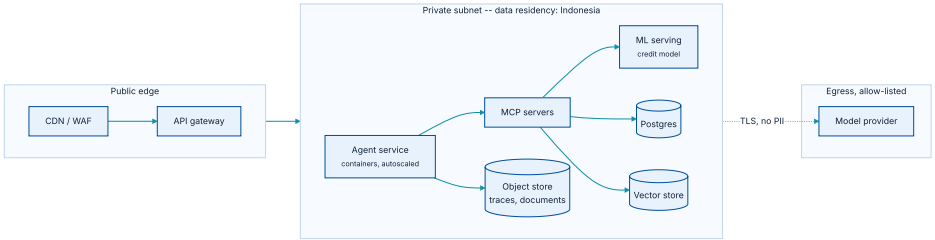


Step 7 is a person. Step 8 is what makes step 7 defensible six months later.

Steps 1–6 are the agent working: read the application, analyse the account, check policy, score, cite the clause, queue the recommendation. **Step 7 is a person**, and nothing in steps 1–6 can reach past it. Step 8 writes the whole run down — **the officer's reason included**, which is the only place this system learns something it did not already know.

DEPLOYMENT

Where each piece runs, and where the data is allowed to be



One egress, allow-listed, carrying no personal data. Everything stateful stays in-region.

The single dotted line is the whole compliance argument. **If that line carries personal data, the design is different** — and you need the contract, the transfer record, and the DPIA to match.

What has to be true before this serves a customer

1. **Egress is allow-listed** and every destination is documented as a transfer.
2. **Redaction is tested**, with a case in the evaluation set that fails if personal data reaches the provider.
3. **Traces are immutable and retained** on a schedule that satisfies both audit and erasure.
4. **Identity propagates** — permissions are evaluated against the officer, never the service account.
5. **Budgets are per tenant**, not only global, so one runaway session cannot exhaust the quota for everyone.
6. **Rollback is rehearsed** for prompts and tool sets, not only for code.
7. **Someone is on call**, and the runbook says what to do when the model provider is degraded.

CLOUD

Providers that can host this, and what to check

PROVIDER	AGENT-RELEVANT SERVICES	INDONESIA REGION
AWS	Bedrock (+ Agents, Guardrails), SageMaker, EKS, Step Functions	Jakarta — ap-southeast-3
Google Cloud	Vertex AI (Agent Builder, Agent Engine), GKE, Workflows	Jakarta — asia-southeast2
Microsoft Azure	Azure AI Foundry, AI Content Safety, AKS, Durable Functions	Indonesia Central
Alibaba Cloud	Model Studio, ACK	Jakarta
Domestic / on-prem	Biznet, Lintasarta, Telkom; or your own data centre with vLLM	By definition

> Region availability is **not** the same as the model being served from that region. Ask specifically where *inference* runs, and get it in the contract.

Six questions a procurement team should carry

- **Where does inference physically run**, and can that be pinned?
- **What is the retention** on prompts and completions — and is zero-retention contractual or a setting someone can flip?
- **Is our data used for training?** Get the default and the opt-out in writing.
- **Which certifications apply** to the specific service, not the provider generally — ISO 27001, 27701, SOC 2, and their scope statements.
- **What is the deprecation policy** for a model version we have validated? A silent model update invalidates your evaluation.
- **What are the exit terms** — can we extract traces, embeddings and fine-tunes, in what format, over what period?

The fifth is the one people miss. **A model updated underneath you is a change you did not test**, and in a regulated setting that is a finding.

11

The demo

A Flutter app on either phone, a REST service, a model behind MCP, and a human who decides.

Why the front end is a phone

CONSTRAINT	WHAT IT FORCES
Poor connectivity	The request is queued and resumable. An agent run is minutes, not milliseconds — treat it as a job, not a call.
Small screen	The recommendation must fit and be scannable. The evidence sits behind a tap, not on the first screen.
Camera as a sensor	Documents arrive as photographs. Extraction happens server-side, with a confirmation step.
Device is not trusted	No credentials for core systems on the phone. It holds a session token and nothing else.

Five screens, and one of them is the point

1 — Applications
What is waiting. The officer picks one and the rest of the app is about that file.

2 — Applicant
The figures, the owner's name — **shown here and carried nowhere else** — and the document camera.

3 — Running
The run is a job. Progress names *which tool is running*, so a slow lookup is distinguishable from a hung one.

4 — Recommendation
Every figure carries its source. Score and reason codes from the credit model, policy clauses printed in full, and the trace one tap away.

5 — Recorded
The officer decides, with a reason that is required even when they agree. The reason is stored against their id.

The screen that is absent
There is no screen, and no endpoint, by which this app disburses anything. The boundary is in the code, not in the navigation.

Screen 3 is where the module's argument becomes a product: **a recommendation you cannot check is a recommendation you should not act on.**

What ships with this module

COMPONENT	STACK	PURPOSE
Mobile app	Flutter — Android + iOS	Five screens from one codebase; the assessment is a job
Agent case	Python, `agentcore`	The loop, budgets, guardrails, traces — from the demo repository
REST service	Python standard library	Stands in for core banking, analytics, policy and the queue. No framework: the point is the shape of the API.
Credit model	scikit-learn	Two candidates fitted, the better one shipped; returns the score **and the reason codes**
MCP server and client	Written out by hand	Line-delimited JSON-RPC. Small enough to read in one sitting, which is why it is not the SDK here.
Approval queue	A locked JSON file	Where a write waits for a person — shared across processes

All of it runs locally with no API key, against the offline provider in `agentcore` — so the whole system can be demonstrated in a classroom with no credentials and no network. **One command runs the lot:** `python3 integrated/run_demo.py –check`.

DEMO · WHAT IT IS DESIGNED TO SHOW

Five things you can only see end to end

1. **An LLM orchestrating, not deciding.** The score comes from a model that can be validated; the LLM chooses what to look up and how to explain it.
2. **A capability boundary that is structural.** No approval tool exists anywhere in the agent's registry.
3. **A trace that is an audit trail.** Every figure in the recommendation traces to the call that produced it.
4. **Redaction that is tested**, with an evaluation case that fails if personal data would reach the provider.
5. **A hand-off that comes back.** The officer's decision and reason return to the system and close the loop.

12

Operating it

How you know it works, and what you do when it stops.

EVALUATION

Three levels, because they catch different things

LEVEL	WHAT IT ASSERTS	WHAT IT MISSES
Unit	The right tool is called with the right arguments	Whether the sequence produces a useful outcome
End to end	The outcome matches a known good one	*Why* — a right answer for the wrong reason still passes
Human review	Whether it is actually good, by a standard you cannot encode	Coverage — it is sampled

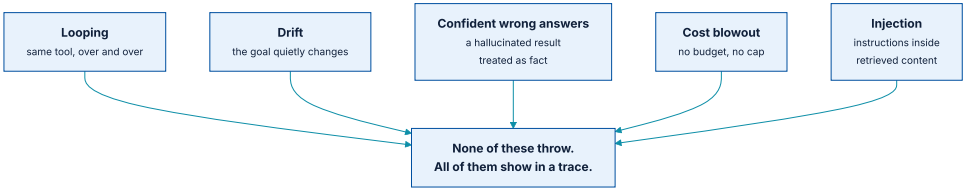
Run all three **before every prompt change, model upgrade, and tool change.** In this domain a prompt edit is a code change with no type checker behind it.

Where the twenty cases come from

1. **Twenty real applications**, taken from what the process handles today. Not twenty imagined ones.
2. **Write the good outcome for each** before building anything, while you are still honest about what good means.
3. **Add every failure you observe**, permanently. This is how the set grows, and why it is versioned.
4. **Include the boring cases.** A set of only hard cases lets you ship something that fails the easy ones.
5. **Keep a held-out slice** you do not look at while iterating — the validation-set overfitting problem, in a new setting.

OPERATING

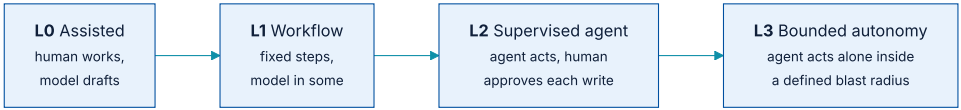
Five failure modes, and how each announces itself



None of these raise an exception. All of them are visible in a trace.

Alert on the **shape** of runs, not only on errors: mean steps per request, cost per request, tool-call repetition rate, escalation rate. **A rising step count is usually the first visible sign that something upstream changed.**

The autonomy ladder, and where to stop



Move up a rung only with evidence from the evaluation harness.

> For a credit decision under OJK supervision, **L2 is the destination**, not a waypoint. The question at L3 is not *how good is it* but **what is the blast radius when it is wrong** — and that boundary belongs in code, not in a prompt.

COMMON FAILURE MODES

Four ways this work goes wrong in practice

Building an agent for a workflow

The most common and most expensive mistake. Apply the operational test first: **who decides the next step?**

Splitting for tidiness

Researcher / Writer / Critic feels organised and costs three times one agent with the same tools. Split on the three questions, not on the org chart.

No trace until the first incident

Nothing here reproduces. Without traces you cannot debug it, and you cannot evidence it to an auditor either.

Compliance as a final gate

The residency question decides the architecture. Asking it after the build means rebuilding.

SUMMARY

What has to survive this module (1 of 2)

1. **The definition is old.** Russell & Norvig: percepts in through sensors, actions out through actuators. What is new is the action space, not the idea.
2. **Who decides the next step** separates a workflow from an agent. Most things are workflows — build them as workflows.
3. **Five workflow patterns come first.** Reach for an agent when the path genuinely cannot be known in advance.
4. **The model never touches anything.** It emits an intention; your code decides. Every permission boundary lives in that gap.
5. **Six exit conditions**, not one. And read tools differ from write tools from day one.
6. **Three questions before splitting** — different permissions, real parallelism, or context that will not fit.

SUMMARY

What has to survive this module (2 of 2)

1. **Components come in three levels**, and calling a minimum system production-ready is how incidents happen.
2. **Prototype in no-code, build in code.** They are different activities; using one tool for both is how prototypes become production by accident.
3. **MCP reaches inward, A2A reaches across.** Adopt the one you need.
4. **Settle the data-residency question first.** It decides the architecture, and it is not an engineering decision.
5. **One control, several obligations.** Traces are the audit trail; evaluation sets are the control tests. Keep the mapping table.
6. **Let the classical model produce the number** and the LLM produce the explanation. Use each for what it can be validated on.

REFERENCES

Where every claim in this module comes from

AREA	PRINCIPAL SOURCES
Foundations	Russell & Norvig, *AI: A Modern Approach*, 4th ed., ch. 2 · Wooldridge, *An Introduction to MultiAgent Systems* · Chollet, *On the Measure of Intelligence* (arXiv:1911.01547)
The loop	ReAct (arXiv:2210.03629) · Chain-of-Thought (arXiv:2201.11903) · Toolformer (arXiv:2302.04761) · Reflexion (arXiv:2303.11366) · CodeAct (arXiv:2402.01030)
Limits	Dziri et al., *Faith and Fate* (arXiv:2305.18654)

REFERENCES

Practice, security, and the rules you are held to

AREA	PRINCIPAL SOURCES
Practice	Anthropic, *Building Effective Agents* (2024) · Model Context Protocol specification · Agent2Agent, Linux Foundation (2025)
Security	OWASP Top 10 for LLM Applications · NIST AI RMF 1.0 (NIST AI 100-1)
Regulation	OJK, *Tata Kelola Kecerdasan Artifisial Perbankan Indonesia* (29 April 2025) · UU 27/2022 (PDP) · POJK 11/POJK.03/2022 · SEOJK 29/SEOJK.03/2022
Standards	ISO/IEC 27001:2022 · ISO/IEC 27701:2025 · ISO/IEC 42001:2023 · ISO/IEC 23894:2023

REFERENCES

The list is a file, and it is checkable

keeping it honest

BASH

```
cd course-slides/references
python3 check.py --pdf      # re-fetch every link, verify every local PDF
```

Two corrections that pass to you. ISO/IEC 27701 is no longer an extension to 27001 — the second edition of 14 October 2025 is standalone. And the OJK AI guidance is dated **2025**, not 2024 as several secondary sources state.

LIST
references/REFERENCES.md

OJK
Tata Kelola Kecerdasan Artifisial Perbankan Indonesia

REPO
ai-agentic-demo

BACK
Large Language Models — Viny's module